

The Gale-Shapley Algorithm (1962): Theory, Analysis, and Applications

Shyamsundar

PGDM in Research and Business Analytics

Madras School of Economics

January 12, 2026

Contents

1	Introduction	2
2	Mathematical Formulation	2
2.1	Definitions and Notation	2
2.2	Matching	3
2.3	Stability and Blocking Pairs	3
2.4	The Main Theorem	3
3	The Gale-Shapley Algorithm (Deferred Acceptance)	3
3.1	Pseudocode	4
4	Hand Calculation and Iteration Trace	4
4.1	Input Preferences	4
4.2	Step-by-Step Iterations	4
4.3	Final Result	6
5	Code of Analysis (Python Implementation)	6
6	Real-World Applications	8
6.1	The National Resident Matching Program (NRMP)	8
6.2	Kidney Paired Donation	8
7	Conclusion	9

1 Introduction

The **Gale-Shapley Algorithm**, formally introduced by David Gale and Lloyd Shapley in their seminal 1962 paper "*College Admissions and the Stability of Marriage*", represents a cornerstone of modern game theory and market design. While initially framed as a theoretical problem regarding stable matching, its implications have reached far beyond abstract mathematics.

In many economic situations, price is not the determining factor for allocation. For example, medical residents cannot simply "buy" a position at a top hospital, nor can public schools auction seats to the highest-bidding parents. These markets are driven by *preferences*, not capital.

The central problem Gale and Shapley addressed was: **How do we match two sets of agents efficiently and stably when they have distinct preferences for one another?**

The significance of this work was recognized globally when Lloyd Shapley was awarded the **Nobel Memorial Prize in Economic Sciences in 2012** (shared with Alvin Roth) for the theory of stable allocations and the practice of market design. This report explores the mathematical foundations, the algorithmic implementation, and the complexity analysis of the Gale-Shapley Deferred Acceptance algorithm.

2 Mathematical Formulation

To analyze the algorithm rigorously, we must first define the problem using set theory and ordinal preferences. We will utilize the context of the **Stable Matching Problem (SMP)** involving Medical Residents and Hospitals.

2.1 Definitions and Notation

Let there be two disjoint sets of size n :

- A set of Medical Residents: $R = \{r_1, r_2, \dots, r_n\}$
- A set of Hospitals: $H = \{h_1, h_2, \dots, h_n\}$

Each agent possesses a strict ordering of preferences over the agents of the opposite set. We denote the preference relation as $\succ$.

- If resident r prefers hospital h_1 over h_2 , we write: $h_1 \succ_r h_2$.
- If hospital h prefers resident r_1 over r_2 , we write: $r_1 \succ_h r_2$.

2.2 Matching

A **matching** M is a one-to-one correspondence (bijection) between the set R and the set H . Formally, M is a set of ordered pairs such that:

$$M \subset R \times H$$

If $(r, h) \in M$, we say that r is matched with h , denoted as $M(r) = h$ and $M(h) = r$.

2.3 Stability and Blocking Pairs

The core constraint of the problem is **stability**. A matching is considered stable only if there is no incentive for any pair of agents to deviate from their assigned match.

Definition (Blocking Pair):

A pair (r, h) is a *blocking pair* for a matching M if:

1. $M(r) \neq h$ (They are not currently matched).
2. $h \succ_r M(r)$ (The resident prefers hospital h over their current assignment).
3. $r \succ_h M(h)$ (The hospital prefers resident r over their current assignment).

Definition (Stable Matching):

A matching M is **stable** if there exist no blocking pairs. In a stable matching, no resident and hospital can mutually improve their situation by abandoning their current partners to join each other.

2.4 The Main Theorem

Gale and Shapley (1962) proved the following fundamental theorem:

"For any equal number of agents with strict preferences, there always exists at least one stable matching."

Furthermore, the Deferred Acceptance algorithm is guaranteed to find such a matching in $O(n^2)$ time.

3 The Gale-Shapley Algorithm (Deferred Acceptance)

The mechanism employed to find the stable matching is known as the **Deferred Acceptance Algorithm**. In this context, we assume the Residents are the "Proposers" and Hospitals are the "Reviewers". This configuration produces a *Resident-Optimal* stable matching.

3.1 Pseudocode

Algorithm 1 Gale-Shapley Resident-Proposing Algorithm

```
1: Initialize all Residents and Hospitals as Free
2: while  $\exists$  a Resident  $r$  who is Free and has not proposed to every Hospital do
3:   Let  $h$  be the highest-ranked Hospital in  $r$ 's preference list to which  $r$  has not yet
   proposed
4:    $r$  proposes to  $h$ 
5:   if  $h$  is Free then
6:      $h$  holds the proposal from  $r$  (tentatively matched)
7:   else
8:     Let  $r_{current}$  be the Resident currently held by  $h$ 
9:     if  $h$  prefers  $r$  over  $r_{current}$  then
10:       $h$  rejects  $r_{current}$  (sets  $r_{current}$  to Free)
11:       $h$  holds the proposal from  $r$  (new tentative match)
12:     else
13:       $h$  rejects  $r$  (remains matched with  $r_{current}$ )
14:     end if
15:   end if
16: end while
17: Output: The set of tentative matches as the Final Stable Matching
```

Key Mechanism: Note that acceptance is never final until the very end. A hospital only says "maybe" (defers acceptance). If a better candidate arrives later, the hospital "trades up," ensuring stability.

4 Hand Calculation and Iteration Trace

To illustrate the algorithm, we consider a system with $n = 3$.

- **Residents:** $R = \{R_1, R_2, R_3\}$
- **Hospitals:** $H = \{H_1, H_2, H_3\}$

4.1 Input Preferences

The following preference lists act as the input for our analysis:

4.2 Step-by-Step Iterations

We trace the algorithm round by round. Residents propose to their top choice that hasn't rejected them yet.

Resident	Preferences (Highest to Lowest)
R_1	H_1, H_2, H_3
R_2	H_2, H_1, H_3
R_3	H_1, H_2, H_3
Hospital	Preferences (Highest to Lowest)
H_1	R_2, R_3, R_1
H_2	R_1, R_2, R_3
H_3	R_1, R_2, R_3

Table 1: Preference Matrix for Simulation

Iteration 1:

- R_1 proposes to H_1 . H_1 is free. $\rightarrow H_1$ **holds** R_1 .
- R_2 proposes to H_2 . H_2 is free. $\rightarrow H_2$ **holds** R_2 .
- R_3 proposes to H_1 .
 - Conflict: H_1 currently holds R_1 but receives a proposal from R_3 .
 - Comparison: H_1 prefers R_3 over R_1 (see preference list).
 - Action: H_1 **rejects** R_1 and **holds** R_3 .

Current State: $(H_1, R_3), (H_2, R_2)$. R_1 is Free.

Iteration 2:

- R_1 (who was rejected) proposes to their next choice: H_2 .
- Conflict: H_2 currently holds R_2 .
- Comparison: H_2 prefers R_1 over R_2 (see preference list).
- Action: H_2 **rejects** R_2 and **holds** R_1 .

Current State: $(H_1, R_3), (H_2, R_1)$. R_2 is Free.

Iteration 3:

- R_2 (who was rejected) proposes to their next choice: H_1 .
- Conflict: H_1 currently holds R_3 .
- Comparison: H_1 prefers R_2 over R_3 .
- Action: H_1 **rejects** R_3 and **holds** R_2 .

Current State: $(H_1, R_2), (H_2, R_1)$. R_3 is Free.

Iteration 4:

- R_3 (who was rejected) proposes to their next choice: H_2 .
- Conflict: H_2 currently holds R_1 .
- Comparison: H_2 prefers R_1 over R_3 .
- Action: H_2 stays with R_1 and **rejects** R_3 .

Current State: $(H_1, R_2), (H_2, R_1)$. R_3 is Free.

Iteration 5:

- R_3 (who was rejected) proposes to their next choice: H_3 .
- H_3 is free. $\rightarrow H_3$ **holds** R_3 .

Current State: $(H_1, R_2), (H_2, R_1), (H_3, R_3)$.

4.3 Final Result

All Residents are matched. The algorithm terminates.

- R_1 is matched with H_2 .
- R_2 is matched with H_1 .
- R_3 is matched with H_3 .

Final Matching set M : $\{(R_1, H_2), (R_2, H_1), (R_3, H_3)\}$

5 Code of Analysis (Python Implementation)

To automate the matching process for larger datasets (e.g., $n > 1000$), we implement the algorithm using Python. The system is designed as a Class-based structure to handle data ingestion, processing, and stability verification.

```
1 class GaleShapley:
2     def __init__(self, residents_pref, hospitals_pref):
3         """
4         Initialize with preference dictionaries.
5         residents_pref: dict {resident: [list of hospitals]}
6         hospitals_pref: dict {hospital: [list of residents]}
7         """
8         self.r_pref = residents_pref
9         self.h_pref = hospitals_pref
10        self.matches = {} # Stores {hospital: resident}
```

```

11     self.free_residents = list(residents_pref.keys())
12
13     def solve(self):
14         # Continue while there is a free resident who still has options
15         while self.free_residents:
16             r = self.free_residents.pop(0)
17             r_list = self.r_pref[r]
18
19             # If resident has run out of hospitals to propose to
20             if not r_list:
21                 continue
22
23             # Propose to the first hospital on the list
24             h = r_list.pop(0)
25
26             if h not in self.matches:
27                 # Hospital is free: Accept immediately
28                 self.matches[h] = r
29                 print(f"{r} proposed to {h} -> Accepted (Free)")
30             else:
31                 # Hospital is occupied: Compare current vs new
32                 current_r = self.matches[h]
33                 h_list = self.h_pref[h]
34
35                 # Check indices (lower index = higher preference)
36                 if h_list.index(r) < h_list.index(current_r):
37                     # New resident is better
38                     self.matches[h] = r
39                     self.free_residents.append(current_r) # Old
resident becomes free
40                     print(f"{r} proposed to {h} -> Accepted (Swap: {
current_r} rejected)")
41                 else:
42                     # Current resident is better
43                     self.free_residents.append(r) # Proposer remains
free
44                     print(f"{r} proposed to {h} -> Rejected")
45
46             return self.matches
47
48     # --- Example Usage ---
49     residents = {
50         'R1': ['H1', 'H2', 'H3'],
51         'R2': ['H2', 'H1', 'H3'],
52         'R3': ['H1', 'H2', 'H3']
53     }
54     hospitals = {

```

```

55     'H1': ['R2', 'R3', 'R1'],
56     'H2': ['R1', 'R2', 'R3'],
57     'H3': ['R1', 'R2', 'R3']
58 }
59
60 system = GaleShapley(residents, hospitals)
61 final_allocation = system.solve()
62 print("\nFinal Stable Matching:", final_allocation)

```

Listing 1: Gale-Shapley Algorithm in Python

Complexity Analysis: The implementation uses a dictionary to store the current state. Since every resident proposes to every hospital at most once, the total number of proposals is bounded by $n \times n$. Thus, the worst-case time complexity is $O(n^2)$.

6 Real-World Applications

The theoretical elegance of the Gale-Shapley algorithm is matched by its practical utility in high-stakes markets.

6.1 The National Resident Matching Program (NRMP)

The most famous application is the matching of medical school graduates to residency programs in the United States.

- **The Problem:** Before 1950, hospitals were competing to sign students earlier and earlier (sometimes 2 years before graduation) to secure talent. This "unraveling" of the market led to inefficient matches.
- **The Solution:** The NRMP was centralized using an algorithm similar to Gale-Shapley.
- **Scale:** Today, the system handles over 40,000 applicants and 30,000 positions annually. The algorithm ensures that the matching is stable, meaning no student and hospital would prefer each other over their assigned matches, thereby preventing "black market" deals.

6.2 Kidney Paired Donation

Alvin Roth adapted these principles to organ donation.

- **The Scenario:** A patient needs a kidney, and their spouse is willing to donate but is biologically incompatible (wrong blood type).

- **The Solution:** The algorithm finds a "cycle" of swaps. Couple A gives a kidney to Couple B, and Couple B gives a kidney to Couple A.
- **Impact:** This application has increased the number of possible transplants significantly, saving thousands of lives.

7 Conclusion

The Gale-Shapley algorithm is a definitive example of how mathematical abstraction can solve human coordination problems. By focusing on **ordinal preferences** rather than cardinal utility (money), it provides a mechanism for **stable** and **strategy-proof** allocation.

While the algorithm guarantees a stable matching, it inherently favors the proposing side (Proposer-Optimal). This insight is crucial for policy design: in the NRMP, the algorithm was adjusted to have students propose to hospitals, rather than vice versa, to favor the students.

Understanding the "Deferred Acceptance" mechanism provides deep insights not just into computer science, but into the hidden structures that govern modern economic markets.

References

- [1] D. Gale and L. S. Shapley, "College Admissions and the Stability of Marriage," *The American Mathematical Monthly*, vol. 69, no. 1, pp. 9-15, 1962.
- [2] A. E. Roth, "The Evolution of the Medical Residency Matching Market," *Journal of Political Economy*, vol. 92, no. 6, pp. 991-1016, 1984.
- [3] The Royal Swedish Academy of Sciences, "The Prize in Economic Sciences 2012: Stable Allocations and the Practice of Market Design," *NobelPrize.org*, 2012.
- [4] J. Kleinberg and E. Tardos, *Algorithm Design*, Pearson Education, 2006.